Lab Manual — Day 6

JavaScript Modules & Tooling — ES Modules, npm, Vite

Instructor	Abid Ali
Duration	90 minutes
Difficulty	Beginner → Intermediate
Prereq	Day 5 lab finished; Node.js v18+ installed
Environment	Node.js v18+ · npm v9+ · a modern browser

Objective

In this lab you will (1) write your first real ES Modules and load them in the browser, (2) initialise an npm project and install a third-party package, and finally (3) scaffold a Vite app, split it into modules, use an npm package inside it, and produce a production build — the exact setup every React / Vue / Svelte project on earth uses.

Timing

Part	Topic	Time
0	Check your toolchain (Node, npm)	5 min
1	Hand-rolled ES Modules in the browser	15 min
2	npm init + install a package + use it in Node	15 min
3	Scaffold a Vite app	10 min
4	Build a multi-module Todo app	25 min
5	Use an npm package inside the Vite app	10 min
6	Production build + inspect /dist	5 min
*	Bonus — aliases, env vars, dynamic import	5 min

Deliverables

- Folder day6-modules-lab/ with TWO sub-folders: part1-raw/ and part2-node/.
- Folder day6-vite-app/ — the scaffolded Vite project with your Todo app inside.
- A short NOTES.md answering the 4 reflection questions at the end.

Part 0 — Check your toolchain (5 min)

Run the following in a fresh terminal. Your versions may be newer, that is fine — they must not be older.

```
node --version    # expect v18 or higher
npm --version     # expect 9 or higher
```

*If Node is missing, install it from **https://nodejs.org** (LTS build).*

Part 1 — Hand-rolled ES Modules (15 min)

export -> import

math.js

```
export const PI = 3.14;
export function add(a,b) {
  return a + b;
}
export default class {...}
// 'public API' of the file
```

→

app.js

```
import { add, PI } from
  './math.js';
import Thing from
  './math.js';
add(2, 3);   // 5
// picks what it needs
```

Each .js file is a MODULE — its own scope. Only exported bindings escape.

You're about to write the left file and the right file.

Task 1.1 — create the project folder

```
mkdir day6-modules-lab
cd day6-modules-lab
mkdir part1-raw
cd part1-raw
```

Task 1.2 — write three files

Create exactly these files:

```
part1-raw/
■■■ index.html
■■■ math.js
■■■ main.js
```

math.js — several named exports and ONE default export.

```
export const PI = 3.14159;

export function add(a, b) {
  return a + b;
}

export function circleArea(r) {
  return PI * r * r;
}

export default class Calculator {
  constructor(start = 0) { this.value = start; }
  add(n)  { this.value += n; return this; }
  mul(n)  { this.value *= n; return this; }
```

```
  result(){ return this.value; }
}
```

main.js — import both named and default.

```
import Calculator, { add, PI, circleArea } from './math.js';

console.log('add(2, 3)      =', add(2, 3));
console.log('PI             =', PI);
console.log('circleArea(5)  =', circleArea(5));

const c = new Calculator(10).add(5).mul(2).result();
console.log('10 + 5, * 2    =', c);

document.body.insertAdjacentHTML(
  'beforeend',
  `<p>circleArea(5) = ${circleArea(5).toFixed(2)}</p>`
);
```

index.html — note type="module".

```
<!doctype html>
<html>
  <head><title>Part 1 — raw modules</title></head>
  <body>
    <h1>Raw ES Modules demo</h1>
    <script type="module" src="./main.js"></script>
  </body>
</html>
```

Task 1.3 — serve it correctly

You **cannot** double-click index.html. ES Modules refuse to load from file://. Start a tiny local server with npm's built-in runner:

```
# from inside part1-raw/
npx serve .
# open the URL it prints (usually http://localhost:3000)
```

Open the browser DevTools → Console. You should see the console.log output, and the page should show "circleArea(5) = 78.54".

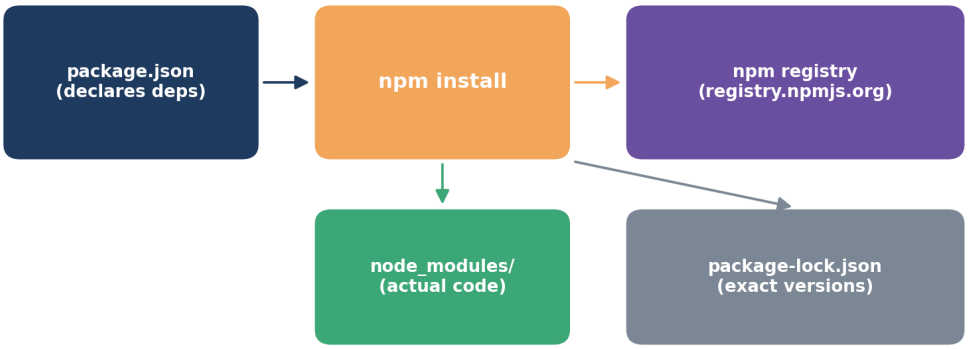
Task 1.4 — reproduce the classic error

Temporarily remove type="module" from the <script> tag and reload. Record the exact error message — you will see this one in the wild. Put it back afterwards.

Checkpoint — you have just written and consumed your first ES Modules, straight in a browser, with no build step at all.

Part 2 — npm init + install a package (15 min)

The npm flow



package.json ↔ registry ↔ node_modules — in action.

Task 2.1 — init the project

```
cd ..                    # back to day6-modules-lab
mkdir part2-node
cd part2-node

npm init -y              # creates package.json with defaults
```

Task 2.2 — open package.json and edit it

Add "type": "module" at the top-level so Node uses ES Modules, and add a start script.

```
{
  "name": "part2-node",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node index.js"
  }
}
```

Task 2.3 — install dayjs

dayjs is a tiny (~2 KB) date library — we'll use it as our guinea-pig package.

```
npm install dayjs
```

Now inspect the results:

- Open package.json again — there is a new dependencies block with dayjs.
- A brand-new package-lock.json exists — it pins the exact version installed.
- A new node_modules/ folder exists — do **not** check this into git.

Task 2.4 — use it

Create index.js:

```
import dayjs from 'dayjs';

const now = dayjs();
console.log('Today is      :', now.format('dddd, MMMM D, YYYY'));
console.log('ISO timestamp :', now.toISOString());
```

```
const bootcampEnd = dayjs('2026-06-30');
console.log(
  'Days until the bootcamp ends:',
  bootcampEnd.diff(now, 'day')
);

npm start
```

Task 2.5 — split into your own module

Create lib/time.js that exports a single named function:

```
// lib/time.js
import dayjs from 'dayjs';

export function daysUntil(isoDate) {
  return dayjs(isoDate).diff(dayjs(), 'day');
}

export function humanDate(d = dayjs()) {
  return dayjs(d).format('dddd, MMMM D, YYYY');
}
```

Now in index.js replace your body with:

```
import { daysUntil, humanDate } from './lib/time.js';

console.log('Today is      :', humanDate());
console.log('Until 2026-06-30:', daysUntil('2026-06-30'), 'days');
```

Re-run npm start. The output should match.

Commit-worthy realisation: you just consumed a third-party package AND built your own reusable module in under 20 lines.

Part 3 — Scaffold a Vite app (10 min)

Task 3.1 — generate the project

```
# from anywhere OUTSIDE day6-modules-lab
npm create vite@latest day6-vite-app

# When prompted:
#   framework -> Vanilla
#   variant   -> JavaScript

cd day6-vite-app
npm install
npm run dev
```

Open the URL it prints (usually http://localhost:5173). You should see Vite's starter page.

Task 3.2 — tour the project

A typical Vite project

```
my-app/
├── index.html
├── package.json
├── package-lock.json
├── vite.config.js
├── node_modules/   ← npm puts installed code here
├── public/         ← static files copied as-is
├── dist/           ← created by `npm run build`
└── src/
    ├── main.js     ← entry point referenced by index.html
    ├── style.css
    └── modules/
        ├── counter.js
        └── api.js
```

Your project should match this shape.

- Open index.html — find the <script type="module" src="/src/main.js"> line.
- Open src/main.js and src/counter.js — these already use real ES Module syntax.
- Edit the word "Hello Vite!" (or similar) in the source, save, and watch the browser update **without a full reload**. That is HMR.

Task 3.3 — first custom change

Replace the default counter's initial value with 42. Save. The browser should reflect the change instantly.

Part 4 — Build a multi-module Todo app (25 min)

You will replace the default demo with a tiny Todo app spread across several files. Practise clean imports and exports.

Task 4.1 — desired final structure

```
day6-vite-app/
■■■ index.html
■■■ package.json
■■■ src/
```

```
    ■■■ main.js
    ■■■ style.css
    ■■■ modules/
        ■■■ store.js       # in-memory todo list
        ■■■ render.js      # draws the UI
        ■■■ events.js      # wires up clicks
```

Task 4.2 — edit index.html body

```
<body>
  <main id="app">
    <h1>Day 6 Todos</h1>
    <form id="todo-form">
      <input id="todo-input" placeholder="What's next?" />
      <button type="submit">Add</button>
    </form>
    <ul id="todo-list"></ul>
  </main>
  <script type="module" src="/src/main.js"></script>
</body>
```

Task 4.3 — src/modules/store.js

```
let todos = [];
let nextId = 1;

export function all()       { return [...todos]; }
export function add(text)   {
  todos.push({ id: nextId++, text, done: false });
}
export function toggle(id)  {
  const t = todos.find(t => t.id === id);
  if (t) t.done = !t.done;
}
export function remove(id)  {
  todos = todos.filter(t => t.id !== id);
}
```

Task 4.4 — src/modules/render.js

```
import { all } from './store.js';

export function render(list) {
  list.innerHTML = all().map(t => `
    <li data-id="${t.id}" class="${t.done ? 'done' : ''}">
      <span class="text">${t.text}</span>
      <button class="del">x</button>
    </li>
  `).join('');
}
```

Task 4.5 — src/modules/events.js

```
import * as store   from './store.js';
import { render }   from './render.js';

export function wire(form, input, list) {
  form.addEventListener('submit', (e) => {
    e.preventDefault();
    const text = input.value.trim();
```

```
    if (!text) return;
    store.add(text);
    input.value = '';
    render(list);
  });

  list.addEventListener('click', (e) => {
    const li = e.target.closest('li');
    if (!li) return;
    const id = Number(li.dataset.id);
    if (e.target.classList.contains('del')) {
      store.remove(id);
    } else {
      store.toggle(id);
    }
    render(list);
  });
}
```

Task 4.6 — src/main.js (entry)

```
import './style.css';
import { wire }   from './modules/events.js';
import { render } from './modules/render.js';

const form  = document.getElementById('todo-form');
const input = document.getElementById('todo-input');
const list  = document.getElementById('todo-list');

wire(form, input, list);
render(list);
```

Task 4.7 — src/style.css

```
body   { font: 16px/1.4 system-ui; padding: 2rem; max-width: 480px; margin: auto; }
h1     { color: #1F3A5F; }
form   { display: flex; gap: .5rem; margin-bottom: 1rem; }
input  { flex: 1; padding: .5rem; font-size: 1rem; }
button { padding: .5rem 1rem; cursor: pointer; }
ul     { list-style: none; padding: 0; }
li     { display: flex; justify-content: space-between;
         padding: .4rem .6rem; border-bottom: 1px solid #eee;
         cursor: pointer; }
li.done .text { text-decoration: line-through; color: #7B8794; }
.del   { border: 0; background: transparent; color: #D9534F; font-weight: bold; }
```

Save. With npm run dev still running you should already have a working todo app.

Part 5 — Use an npm package in your Vite app (10 min)

Task 5.1 — install dayjs

```
npm install dayjs
```

Task 5.2 — timestamp every todo

Edit store.js to record a creation timestamp:

```
// at the top
import dayjs from 'dayjs';

export function add(text) {
  todos.push({
    id: nextId++,
    text,
    done: false,
    createdAt: dayjs().format('HH:mm'),
  });
}
```

Then update render.js to display the timestamp:

```
list.innerHTML = all().map(t => `
  <li data-id="${t.id}" class="${t.done ? 'done' : ''}">
    <span class="text">${t.text}
      <small style="color:#7B8794">· ${t.createdAt}</small>
    </span>
    <button class="del">x</button>
  </li>
`).join('');
```

Add a todo — it should show the time next to the text.

Part 6 — Production build (5 min)

Vite — two modes, one tool

vite (dev)

- Native ES modules in the browser
- Instant server start (no bundling)
- HMR — edit file, see change
- Pre-bundles deps with esbuild

npm run dev

-> http://localhost:5173

vite build (prod)

- Rollup under the hood
- Tree-shaken, minified bundle
- Hashed filenames for caching
- Outputs to /dist

npm run build

npm run preview

Dev mode we've used so far. Now it's time for build mode.

```
npm run build
# check what it produced:
ls dist

npm run preview
# open the URL it prints — this is the OPTIMISED bundle
```

- Open dist/index.html — notice the hashed main-XxXxXx.js filename.
- Open the hashed JS file — the code is minified on one line. That's your Todo app bundled and ready to upload anywhere.
- The whole dist/ folder is what a deploy to Netlify, Vercel, GitHub Pages, S3 or any static host would contain.

Bonus — Power-user tricks (5 min)

Bonus A — Import alias

Create a vite.config.js in the project root so @/modules/store.js maps to src/modules/store.js:

```
// vite.config.js
import { defineConfig } from 'vite';
import path from 'path';

export default defineConfig({
  resolve: {
    alias: {
      '@': path.resolve(process.cwd(), 'src'),
    },
  },
});
```

Then in main.js switch to import { wire } from '@/modules/events.js';.

Bonus B — Environment variables

Create a .env file in the project root:

```
# .env
VITE_APP_TITLE="My Day 6 Todos"
```

Variables must start with VITE_ to be exposed to the client. Read them via import.meta.env:

```
document.querySelector('h1').textContent =
  import.meta.env.VITE_APP_TITLE;
```

Bonus C — Dynamic (lazy) import

Suppose formatting is only needed when the user hovers the first todo. Load dayjs on demand and watch the Network tab split into a second chunk in production:

```
list.addEventListener('mouseenter', async () => {
  const { default: dayjs } = await import('dayjs');
  console.log('Lazy-loaded dayjs at', dayjs().toISOString());
}, { once: true });
```

Submission checklist

- day6-modules-lab/part1-raw/ loads correctly via npx serve . and logs to the console.
- day6-modules-lab/part2-node/ runs with npm start and imports dayjs through your own lib/time.js module.
- day6-vite-app/ runs via npm run dev; the Todo app add / toggle / delete works.
- npm run build succeeds and dist/ exists with hashed filenames.
- node_modules/ is NOT committed (add it to .gitignore — Vite already does this for you).

Reflection questions — answer in NOTES.md

- Why did Part 1 need npx serve . and not just double-click index.html?
- What does "type": "module" in package.json change about how Node loads your files?
- Why is committing node_modules/ to git a bad idea — and what guarantees reproducibility instead?
- Name ONE thing Vite does at build time that the browser couldn't do on its own.

Self-assessment rubric

Level	What it looks like
Pass	Parts 0–3 complete; raw modules load, dayjs runs in Node, Vite dev server runs.
Good	All of Pass + Todo app works in Vite + production build succeeds.
Great	All of Good + Part 5 timestamps + at least one Bonus completed.

You just set up the exact workflow that every React / Vue / Svelte project in the world uses. From tomorrow onwards, we take this project structure as a given. Nice work!